



**Technische Hochschule Deggendorf (THD) –
Campus Cham**

Faculty of Applied Natural Sciences and Industrial Engineering
Master Applied AI for Digital Production Management

Case Study: Intelligent Systems in Production (WS25/26)

Predictive Maintenance of Milling Machines using AI/ML

Final Written Report

Team Members	Matriculation No.
Usama Ali	22300108
Irtaza Janjua	12404930
Sadeed Ahmad	22301000
Soman Tariq	22310003

15 January 2026

Abstract

Predictive Maintenance (PdM) has become an important component of intelligent production systems within the Industry 4.0 framework. Traditional maintenance strategies, such as reactive or strictly time-based approaches, often lead to unexpected machine downtime, higher operational costs, and inefficient use of critical assets, particularly milling machines. In this project, we developed and evaluated an AI-based PdM system that estimates the likelihood of machine failure using sensor-derived operational data.

To address the challenge of rare and complex failure patterns, we implemented a deep Artificial Neural Network (ANN) and compared its performance with a Random Forest model serving as an interpretable baseline. The models were trained and evaluated using the AI4I 2020 Predictive Maintenance dataset, which contains 10,000 samples. The workflow included data cleaning, label consistency checks, and mitigation of severe class imbalance through SMOTE, followed by evaluation using standard classification metrics.

Beyond offline modeling, we focused on practical deployment aspects by building a reproducible inference pipeline and a Streamlit-based dashboard. The application supports both single-instance predictions and batch inference from CSV files, using saved preprocessing and model artifacts to ensure consistent behavior. The results show that the ANN improves failure detection performance, particularly in terms of recall for the failure class, while maintaining strong overall accuracy. Overall, this project demonstrates an end-to-end PdM solution that combines reliable modeling with realistic deployment considerations for smart manufacturing environments.

Acknowledgements

We thank the course instructor **Dr. Hamidreza Heidari** and **Technische Hochschule Deggendorf (Campus Cham)** for providing the academic framework and feedback throughout the project.

Contents

1	Introduction	1
1.1	Background	1
1.2	Predictive Maintenance in Industry 4.0	1
1.3	Motivation and Industrial Relevance	1
1.4	Project Objectives	2
1.5	Report Structure	2
1.6	Report Structure	2
2	Problem Statement and Measurable Outcomes	3
2.1	Problem Definition	3
2.2	Scope of the Project	3
2.3	Measurable Outcomes (KPIs)	4
3	Literature Review	5
3.1	Machine Learning Methods for Predictive Maintenance	5
3.2	Deep Learning in PdM	5
3.3	Explainable AI (XAI) Considerations	6
3.4	Class Imbalance in Predictive Maintenance	6
3.5	Research Gap and Project Contribution	6
4	Dataset Description	7
4.1	Dataset Overview	7
4.2	Feature Description	7
4.3	Target Variable	8
5	Data Preprocessing and Exploratory Analysis	10
5.1	Data Cleaning and Consistency Corrections	10
5.2	Train/Test Splitting and Leakage Prevention	10
5.3	Feature Scaling	11
5.4	Handling Severe Class Imbalance with SMOTE	11
5.5	Exploratory Analysis using PCA	11

6	Modeling and Implementation	13
6.1	Overall System Architecture	13
6.2	Baseline Model: Random Forest	14
6.2.1	Rationale	14
6.2.2	Implementation Summary	14
6.2.3	Expected Strengths and Limitations	14
6.3	Proposed Model: Deep Artificial Neural Network (ANN)	15
6.3.1	Rationale	15
6.3.2	Architecture	15
6.3.3	Training Strategy	15
6.4	Reproducible Inference Pipeline and Saved Artifacts	16
6.4.1	Code Structure for Deployment	16
6.4.2	Key Inference Logic	17
7	Results, Evaluation, and Validation	18
7.1	Evaluation Metrics	18
7.2	Baseline vs. ANN Comparison	18
7.2.1	Confusion Matrices	18
7.2.2	ROC Curves	19
7.2.3	Performance Summary Table	20
7.3	Validation Strategy	20
7.4	Benchmarking with Literature	21
8	Deployment: Streamlit User Interface and Batch Inference	22
8.1	Deployment Goals	22
8.2	User Interface Overview	22
8.3	Single Prediction Workflow	24
8.4	Batch Prediction via CSV Upload	26
8.5	Operational Thresholds and Decision Support	27
8.6	How to Run the Application (Reproducibility)	27
9	Discussion, Limitations, Risks, and Ethics	28
9.1	Key Findings	28
9.2	Industrial Interpretation of Results	28
9.3	Limitations	28
9.4	Risks and Mitigation	29
9.5	Ethical Considerations	29
9.6	Future Work	29
10	Conclusion	30

A	Appendix A: Figures Checklist (Where to Insert Which Graph)	31
B	Appendix B: Example CSV Schema for Batch Prediction	32
C	Appendix C: Repository and Reproducibility Notes	33

List of Figures

4.1	Distribution of the five key sensor readings, separated by failure (1) and no-failure (0) cases. The plots reveal overlapping patterns between the two classes, highlighting the challenge of detecting failures from noisy sensor data.	8
4.2	Distribution of machine types in the dataset, showing the proportion of low (L), medium (M), and high (H) quality machines.	9
4.3	The original dataset’s class distribution of machine failures highlights the stark disparity between failure and no-failure samples as well as the unequal distribution among various failure categories.	9
5.1	Class distribution comparison before and after applying SMOTE. The original dataset shows a strong imbalance, with failures severely underrepresented. After oversampling, the dataset becomes more balanced—giving the model a better chance to learn from rare failure cases.	11
5.2	Two-dimensional projection of the dataset using the first two principal components (PC1 and PC2). Points are colored by failure status, showing a partial separation between failure and non-failure cases. The plot also highlights the underlying variance structure captured by these components.	12
5.3	Loadings for the first three principal components (PC1–PC3). PC1 is largely influenced by air and process temperature, PC2 reflects the relationship between torque and speed, and PC3 is shaped mainly by tool wear. These patterns support the interpretability of the components in the context of machine behavior.	12
6.1	System architecture of the deployed predictive maintenance solution. Raw sensor data is first processed through a preprocessing pipeline—handling encoding, scaling, and feature alignment—before being passed to the model inference stage. Both Random Forest and ANN models are supported. The final output is presented to users through a Streamlit interface, which enables real-time or batch prediction workflows.	14

6.2	Architecture of the deep feed-forward ANN used in this project. The network processes six sensor features through a series of dense layers (512 → 32), with ReLU activations, batch normalization, dropout, and L2 regularization applied to improve generalization. The final output layer produces a failure probability score, which supports decision-making during deployment.	15
6.3	Training curves for the proposed ANN model. The plots display both training and validation loss (left) and accuracy (right). The model shows good generalization performance, with validation metrics tracking training metrics closely. Learning-rate scheduling and early stopping helped stabilize training and avoid overfitting.	16
6.4	Directory structure used for deployment. It includes the Streamlit app (<code>app.py</code>), inference module (<code>inference.py</code>), serialized model artifacts (<code>artifacts/</code>), and the project's dependency file (<code>requirements.txt</code>).	17
7.1	Confusion matrix for the Random Forest model on the test set. While it shows strong overall performance, several failure cases are misclassified as normal.	19
7.2	Confusion matrix for the ANN model on the test set. The model captures more true failures, with improved recall over the Random Forest baseline.	19
7.3	ROC curves for the ANN and Random Forest models on the test set. The ANN achieves a higher Area Under the Curve (AUC), indicating superior classification performance across different thresholds.	20
8.1	Main screen of the Streamlit dashboard. Users can enter individual sensor values, adjust the decision threshold, or upload a CSV file for batch prediction. Results are presented interactively and can be downloaded for further analysis.	23
8.2	Result screen for a single prediction. The dashboard displays the computed failure probability and a binary decision (Fail / No Fail) based on the user-defined threshold.	25
8.3	Streamlit batch prediction interface showing CSV upload, automated preprocessing, tabular prediction results, and the option to download the annotated output file.	26

List of Tables

4.1	Input Features Used for Predictive Maintenance Modeling	8
7.1	Performance Summary on Test Set	20
7.2	Contextual benchmarking against selected predictive maintenance literature. Due to differences in datasets and experimental setups, comparisons are indicative only.	21

Chapter 1

Introduction

1.1 Background

Data-driven decision-making, continuous sensing, and interconnected machines have become defining characteristics of modern production environments. In such settings, milling machines represent critical assets, particularly in high-mix and high-utilization operations. Even minor, unexpected failures can quickly propagate into production delays, quality issues, and increased operational costs.

1.2 Predictive Maintenance in Industry 4.0

Maintenance strategies in industrial systems are commonly categorized as reactive (run-to-failure), preventative (time-based), and predictive (condition-based). Predictive Maintenance (PdM) focuses on estimating the likelihood of future failures by monitoring measurable machine condition signals during operation. This enables maintenance actions to be planned before critical breakdowns occur. Within the Industry 4.0 paradigm, PdM is typically implemented using industrial IoT sensors, data pipelines, and AI/ML models that analyze operational data to identify early signs of degradation.

1.3 Motivation and Industrial Relevance

Unplanned downtime remains one of the most costly challenges in manufacturing operations. From an industrial perspective, the ability to anticipate failures offers several tangible benefits, including:

- reducing unexpected downtime and associated production losses,
- improving planning of spare parts inventory and maintenance personnel,

- increasing equipment utilization and overall system reliability,
- supporting stable processes and consistent product quality.

Motivated by these considerations, this project aims to translate predictive maintenance concepts into a practical AI-based PdM system tailored to milling machine applications.

1.4 Project Objectives

The main objectives of this project are as follows:

- To design and evaluate an AI-based predictive maintenance model capable of estimating machine failure probabilities from real-time sensor data.
- To benchmark a traditional, interpretable method (Random Forest) against a deep learning approach (ANN) in order to assess trade-offs between accuracy and explainability.
- To develop a fully reproducible inference pipeline, including standardized pre-processing steps (e.g., feature ordering, scaling, type mapping) and deployment-ready model artifacts.
- To create an interactive dashboard using Streamlit that enables both real-time single-instance predictions and batch processing of uploaded data—providing a practical demonstration of the system’s industrial applicability.

1.5 Report Structure

1.6 Report Structure

This report is organized into nine chapters, each addressing a specific aspect of the project. Chapter 2 introduces the problem and outlines the key performance indicators used to measure results. Chapter 3 reviews related work and prior research in predictive maintenance and machine learning. The dataset used for model development is described in Chapter 4, followed by data preprocessing and exploratory analysis in Chapter 5. Chapter 6 provides a detailed explanation of the modeling process, including implementation of both baseline and deep learning models. Chapter 7 presents the evaluation results and discusses model performance. The deployment strategy and user interface, developed using Streamlit, are covered in Chapter 8. Chapter 9 reflects on the system’s limitations, addresses ethical considerations, and proposes directions for future work. The report concludes with a summary of findings and a list of references.

Chapter 2

Problem Statement and Measurable Outcomes

2.1 Problem Definition

Failures in milling machines can lead to a range of operational and financial disruptions, particularly in high-demand production environments. The most common consequences include:

- production delays and missed delivery deadlines,
- increased maintenance costs due to unplanned repairs and urgent spare part replacements,
- lower overall equipment effectiveness (OEE) resulting from underutilization,
- compromised product quality when machine wear or degradation goes unnoticed.

To address these challenges, this project aims to develop a predictive maintenance system that estimates the likelihood of machine failure based on real-time operational data. By identifying early warning signs of failure, t

2.2 Scope of the Project

This project focuses on the development and deployment of predictive maintenance models using a publicly available dataset. Both classical machine learning and deep learning approaches are explored and compared to understand their effectiveness in failure prediction scenarios.

The final system delivers the following key components:

- a trained predictive model capable of estimating failure likelihood from sensor data,
- a reproducible inference pipeline that includes saved preprocessing steps and model artifacts,
- an interactive Streamlit dashboard that allows users to perform real-time predictions and batch scoring from uploaded CSV files.

2.3 Measurable Outcomes (KPIs)

While accuracy is a commonly used metric in classification tasks, it can be misleading in the context of predictive maintenance, where failures are rare but carry high consequences. Therefore, the evaluation of our models relies on a more comprehensive set of performance indicators, including:

- **Failure Recall (Sensitivity):** emphasizes the system's ability to correctly identify failure cases, helping minimize costly false negatives,
- **Precision:** ensures that the model doesn't trigger unnecessary alarms by confirming that positive predictions are likely to be correct,
- **F1-score:** offers a balanced view by combining precision and recall, especially useful in imbalanced datasets,
- **ROC-AUC:** reflects the model's ability to distinguish between failure and non-failure cases across various thresholds,
- **Deployment readiness:** evaluates how reliably the model performs in real-world scenarios, including proper preprocessing, input validation, and integration into the dashboard.

Chapter 3

Literature Review

3.1 Machine Learning Methods for Predictive Maintenance

A variety of machine learning techniques have been applied to predictive maintenance tasks across different industrial domains. Commonly used algorithms include Random Forests, Support Vector Machines (SVMs), boosting methods, and neural networks, each offering distinct strengths depending on the application context [1].

Among these, Random Forests are especially popular due to their robustness, ability to model nonlinear relationships, and ease of interpretability through feature importance rankings [2]. Their relatively low sensitivity to parameter tuning and resistance to overfitting make them a reliable baseline in many PdM systems.

3.2 Deep Learning in PdM

Deep learning models are well-suited for capturing complex, nonlinear relationships within sensor data—making them valuable tools for identifying subtle or rare failure patterns in machinery. Their strength lies in their ability to model intricate interactions that may not be easily detected by traditional algorithms. This is particularly useful in industrial settings where failure indicators may be sparse or buried within noisy operational data.

However, the flexibility of deep models comes with challenges. They are prone to overfitting, especially in imbalanced datasets, and therefore require careful regularization and rigorous validation to ensure generalization.

3.3 Explainable AI (XAI) Considerations

In real-world PdM applications, model transparency is critical. Maintenance teams must be able to understand and trust the system’s predictions—especially when those decisions lead to costly interventions. Explainable AI (XAI) techniques address this need by providing insights into why a model made a certain prediction, supporting human-in-the-loop decision-making. Prior studies have emphasized the importance of interpretable outputs in PdM systems, particularly when deployed in environments with safety or financial implications [3].

3.4 Class Imbalance in Predictive Maintenance

In most predictive maintenance datasets, failure cases are rare compared to normal operating conditions, resulting in a significant class imbalance. This skew can lead to biased models that favor the majority class while failing to identify critical failure events. To address this, techniques such as SMOTE (Synthetic Minority Over-sampling Technique) are commonly used to generate synthetic examples of the minority class and improve the model’s sensitivity to rare events [4].

This step is especially important in PdM applications, where missing a potential failure (false negative) can lead to unplanned downtime and significant costs—often far more than a false alarm would.

3.5 Research Gap and Project Contribution

While many previous studies demonstrate high model accuracy in offline settings, relatively few address whether these models are suitable for deployment in real industrial environments. This project aims to bridge that gap by delivering a fully operational predictive maintenance pipeline that covers data preprocessing, model training and evaluation, and a real-time user interface built with Streamlit. The system supports both single-instance and batch predictions, emphasizing practical usability in addition to modeling performance.

Chapter 4

Dataset Description

4.1 Dataset Overview

The modeling in this project is based on the AI4I 2020 Predictive Maintenance dataset, available from the UCI Machine Learning Repository [5]. It includes 10,000 labeled records collected from simulated milling machine operations. Each entry represents a snapshot of sensor readings taken during operation, along with a label indicating whether a machine failure occurred.

The dataset combines several continuous sensor measurements with one categorical feature that reflects machine quality. This mix of data types allows for experimentation with both traditional machine learning models and more complex deep learning approaches.

4.2 Feature Description

Table 4.1 provides a summary of the input features used for training and evaluating the predictive maintenance models.

Table 4.1: Input Features Used for Predictive Maintenance Modeling

Feature	Description
Machine Type (L/M/H)	Categorical indicator of machine quality category (Low / Medium / High).
Air Temperature (K)	Ambient air temperature near the machine.
Process Temperature (K)	Process temperature during operation.
Rotational Speed (rpm)	Spindle speed during milling operation.
Torque (Nm)	Measured torque during operation.
Tool Wear (min)	Tool wear accumulation indicator.

4.3 Target Variable

The primary prediction target in this project is a binary label indicating whether a machine failure occurred (1) or not (0). The dataset encompasses several types of failure events, including tool wear failure, power failure, overstrain, heat dissipation issues, and random system failures. While these categories are not modeled separately, they are collectively treated as indicators of a general failure state.

During deployment, the system is designed to output a failure probability score for each input instance. This probability serves as decision support for maintenance teams, helping prioritize inspections or interventions before a breakdown occurs.

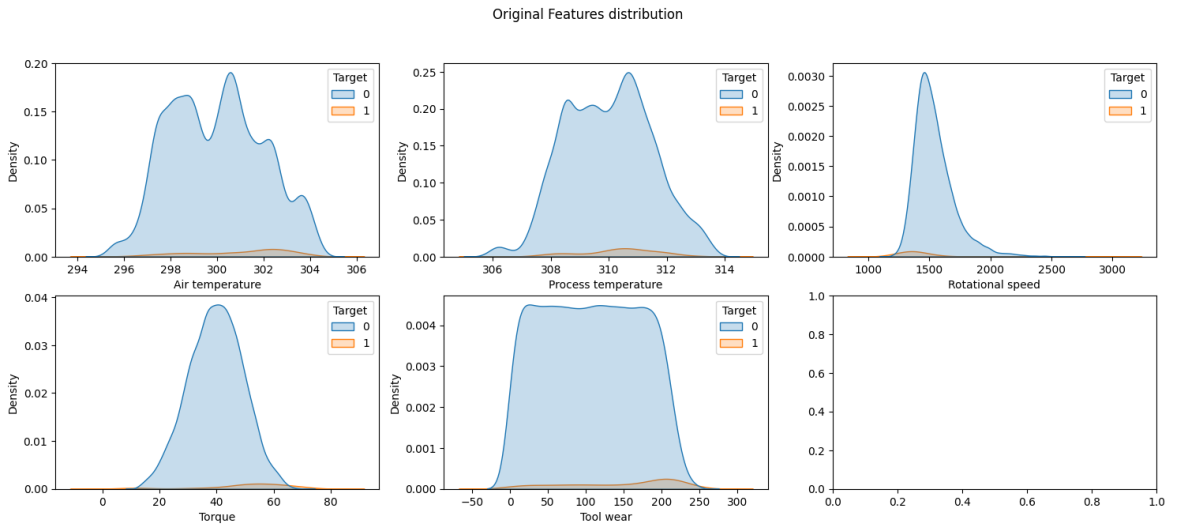


Figure 4.1: Distribution of the five key sensor readings, separated by failure (1) and no-failure (0) cases. The plots reveal overlapping patterns between the two classes, highlighting the challenge of detecting failures from noisy sensor data.

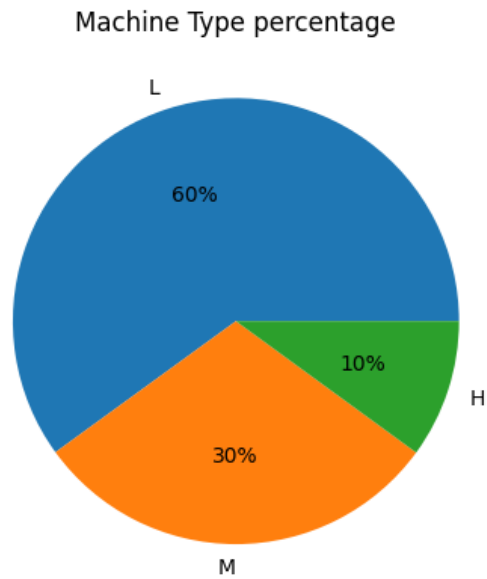


Figure 4.2: Distribution of machine types in the dataset, showing the proportion of low (L), medium (M), and high (H) quality machines.

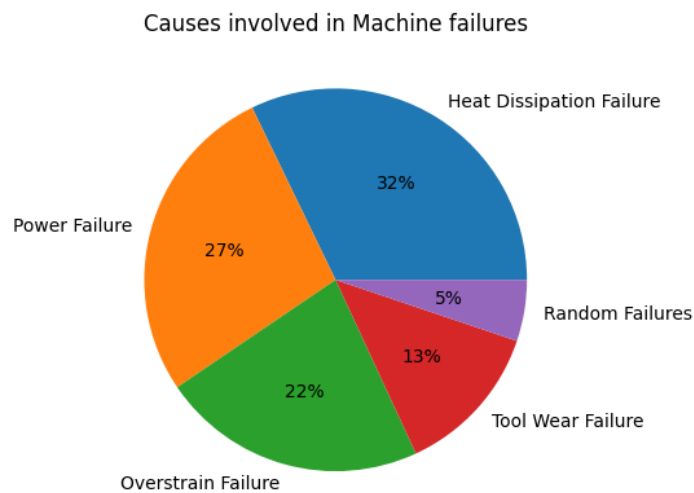


Figure 4.3: The original dataset's class distribution of machine failures highlights the stark disparity between failure and no-failure samples as well as the unequal distribution among various failure categories.

Chapter 5

Data Preprocessing and Exploratory Analysis

5.1 Data Cleaning and Consistency Corrections

Before training the models, several preprocessing steps were applied to ensure the dataset was clean, reliable, and suitable for machine learning tasks:

- Columns containing unique identifiers (such as row-level IDs) were removed, as they do not generalize and may introduce noise.
- Data types were verified and corrected where needed, including consistent encoding of categorical features.
- Label inconsistencies were addressed—particularly for the Random Failure (RNF) category—to ensure that failure types were properly aligned with the binary failure target.
- A small number of ambiguous records were excluded, specifically those where a failure was recorded but no corresponding failure type was listed.

5.2 Train/Test Splitting and Leakage Prevention

To fairly evaluate model performance, the dataset was split into separate training and test sets. Care was taken to avoid data leakage: any transformation that involved learning from data (such as feature scaling or class balancing via SMOTE) was applied only on the training set. The resulting transformations were then applied to the test set to maintain consistency during inference.

5.3 Feature Scaling

All numerical features were standardized using a fitted scaler to bring them onto a common scale. This step helps many machine learning algorithms perform more effectively, particularly those sensitive to feature magnitudes. The trained scaler was saved as a serialized artifact (e.g., `scaler.joblib`) and reused during inference to ensure that the input data is processed in the same way during deployment.

5.4 Handling Severe Class Imbalance with SMOTE

In the original dataset, machine failures account for only a small fraction of the total records. This imbalance can lead to models that default to predicting the majority class—correctly labeling most normal cases while missing rare but critical failures.

To address this, the SMOTE (Synthetic Minority Over-sampling Technique) algorithm was applied during training to generate synthetic examples of the minority (failure) class [4]. This helps the model learn meaningful patterns associated with failures, improving its ability to detect them during inference.

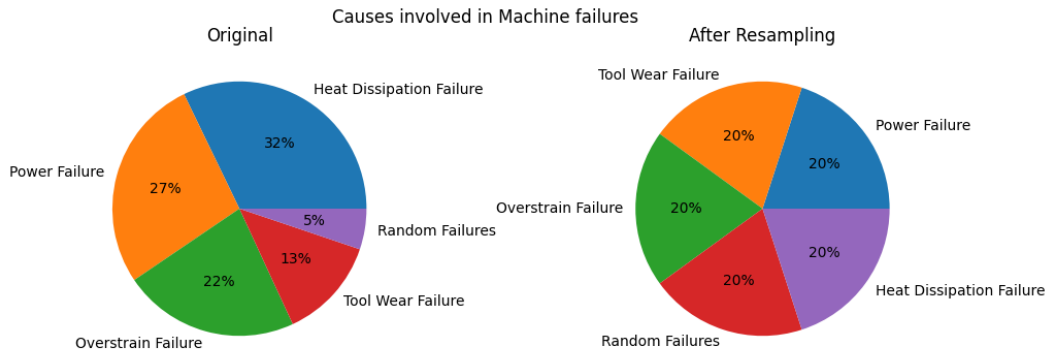


Figure 5.1: Class distribution comparison before and after applying SMOTE. The original dataset shows a strong imbalance, with failures severely underrepresented. After oversampling, the dataset becomes more balanced—giving the model a better chance to learn from rare failure cases.

5.5 Exploratory Analysis using PCA

To better understand the structure of the dataset and the relationships among features, Principal Component Analysis (PCA) was applied as an exploratory step. The first few principal components revealed patterns consistent with the physical behavior of the milling process:

- PC1 appears to be primarily influenced by temperature-related features,

- PC2 captures interactions between rotational speed and torque, suggesting power-related effects,
- PC3 is closely associated with tool wear accumulation.

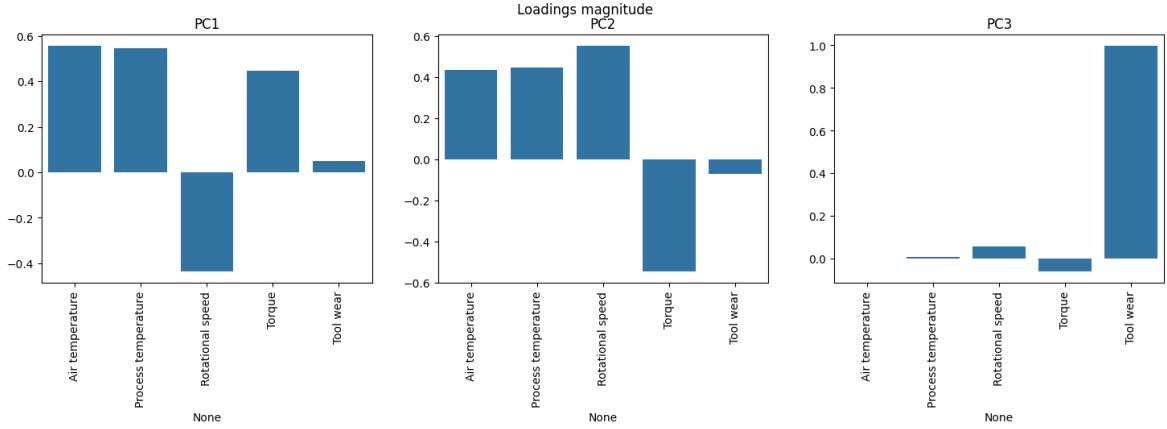


Figure 5.2: Two-dimensional projection of the dataset using the first two principal components (PC1 and PC2). Points are colored by failure status, showing a partial separation between failure and non-failure cases. The plot also highlights the underlying variance structure captured by these components.

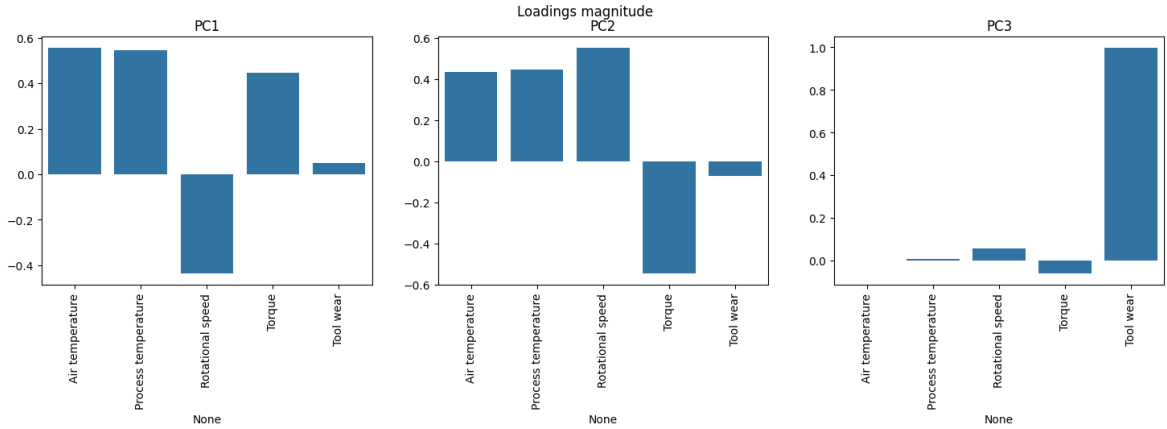


Figure 5.3: Loadings for the first three principal components (PC1–PC3). PC1 is largely influenced by air and process temperature, PC2 reflects the relationship between torque and speed, and PC3 is shaped mainly by tool wear. These patterns support the interpretability of the components in the context of machine behavior.

Chapter 6

Modeling and Implementation

6.1 Overall System Architecture

The complete predictive maintenance system integrates data processing, model inference, and a user-facing interface. Its main components include:

- a preprocessing pipeline that performs encoding, feature scaling, and consistent feature ordering,
- two inference models: a Random Forest baseline and a deep Artificial Neural Network (ANN),
- a deployment interface built using Streamlit, supporting both single-instance and batch CSV-based predictions,
- a set of saved artifacts—including the trained ANN model, preprocessing scaler, feature mappings, and ordering definitions—for consistent inference at runtime.

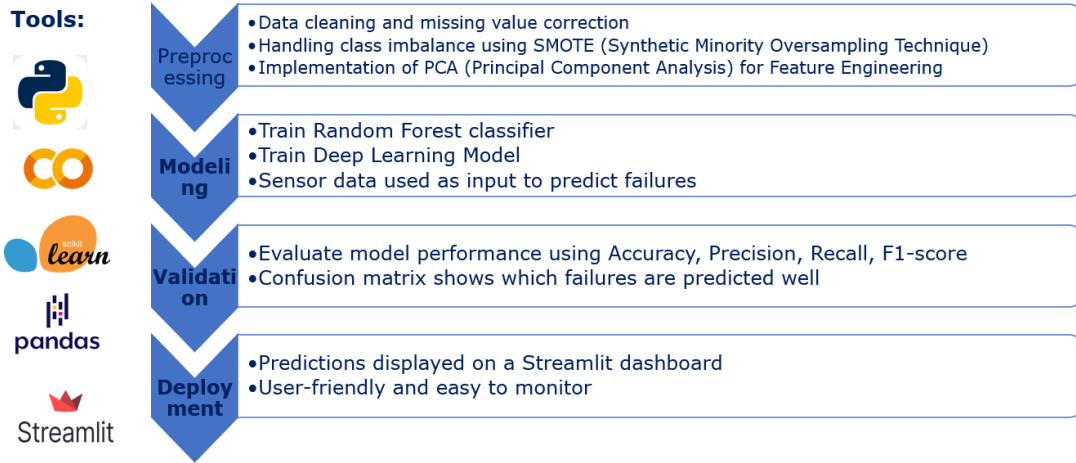


Figure 6.1: System architecture of the deployed predictive maintenance solution. Raw sensor data is first processed through a preprocessing pipeline—handling encoding, scaling, and feature alignment—before being passed to the model inference stage. Both Random Forest and ANN models are supported. The final output is presented to users through a Streamlit interface, which enables real-time or batch prediction workflows.

6.2 Baseline Model: Random Forest

6.2.1 Rationale

Random Forests are a popular choice in predictive maintenance applications due to their robustness, ability to capture nonlinear relationships, and interpretability [2]. In particular, they offer feature importance rankings that can help maintenance engineers identify which sensor readings most influence failure predictions.

6.2.2 Implementation Summary

The model was trained on the preprocessed dataset, with hyperparameters such as the number of trees and maximum depth tuned to balance performance with generalization. As a baseline, the Random Forest provides a solid benchmark against which more complex models can be compared.

6.2.3 Expected Strengths and Limitations

Strengths: High baseline accuracy, interpretable results, and low computational overhead.

Limitations: May underperform on subtle or complex patterns, especially in rare failure cases, where deeper models can offer improved sensitivity.

6.3 Proposed Model: Deep Artificial Neural Network (ANN)

6.3.1 Rationale

Artificial Neural Networks (ANNs) are well-suited for learning complex, nonlinear patterns that might be missed by traditional algorithms. In the context of predictive maintenance, they can uncover intricate relationships between sensor variables and failure modes, which is particularly valuable when dealing with noisy or sparse data.

6.3.2 Architecture

The proposed model is a deep feed-forward ANN composed of multiple fully connected layers. The final configuration includes:

- Five hidden layers with decreasing widths: $512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$
- ReLU activation functions for non-linearity
- Batch normalization to stabilize training and improve convergence
- Dropout and L2 regularization to reduce overfitting
- A sigmoid or softmax output layer, depending on binary vs. multi-class task (this project uses the binary failure probability output)

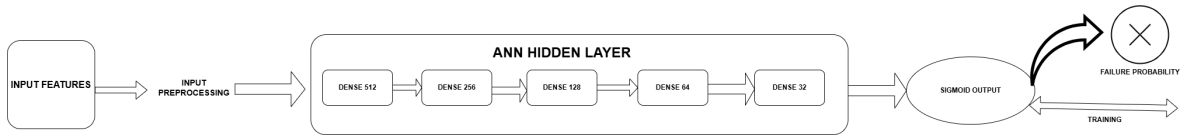


Figure 6.2: Architecture of the deep feed-forward ANN used in this project. The network processes six sensor features through a series of dense layers ($512 \rightarrow 32$), with ReLU activations, batch normalization, dropout, and L2 regularization applied to improve generalization. The final output layer produces a failure probability score, which supports decision-making during deployment.

6.3.3 Training Strategy

The ANN was trained using the Adam optimizer, which balances speed and stability in weight updates. To prevent overfitting and promote efficient convergence, two callbacks were employed: early stopping (to halt training when validation performance stopped improving) and learning-rate reduction on plateau (to slow learning if progress stalled).

The network was trained using a batch size of 32, with a maximum epoch limit of 200. In practice, training often completed earlier due to early stopping, once optimal validation performance was reached.



Figure 6.3: Training curves for the proposed ANN model. The plots display both training and validation loss (left) and accuracy (right). The model shows good generalization performance, with validation metrics tracking training metrics closely. Learning-rate scheduling and early stopping helped stabilize training and avoid overfitting.

6.4 Reproducible Inference Pipeline and Saved Artifacts

To ensure the trained model behaves consistently during deployment, several components from the training process are saved and reused in the inference pipeline:

- A fitted scaler for numerical features, saved to standardize inputs during inference,
- A JSON file defining the exact feature order to avoid mismatched column positions,
- A mapping file for machine type encoding (L/M/H), ensuring consistent representation of categorical inputs,
- The trained ANN model, saved as a serialized Keras artifact for inference use.

6.4.1 Code Structure for Deployment

The project is organized to support easy testing, modification, and deployment. Key components include:

- `app.py`: the Streamlit-based user interface for real-time and batch predictions,
- `inference.py`: core logic for preprocessing input data and generating model predictions,
- `artifacts/`: a directory containing saved model weights, preprocessing tools, and configuration files,

- `requirements.txt`: a list of Python dependencies required to run the application.

```
PS D:\Predictive_Maintenance_Project> ls

Directory: D:\Predictive_Maintenance_Project

Mode                LastWriteTime         Length Name
----                -
d-----          1/11/2026   2:25 PM             artifacts
d-----          1/11/2026   2:35 PM             venv
d-----          1/11/2026   2:28 PM            __pycache__
-a-----          1/11/2026   2:17 PM          6679 app.py
-a-----          1/11/2026   2:17 PM          1879 inference.py
-a-----          1/11/2026   2:17 PM           54 requirements.txt

PS D:\Predictive_Maintenance_Project> |
```

Figure 6.4: Directory structure used for deployment. It includes the Streamlit app (`app.py`), inference module (`inference.py`), serialized model artifacts (`artifacts/`), and the project’s dependency file (`requirements.txt`).

6.4.2 Key Inference Logic

The following code snippet highlights the main steps in the inference pipeline. These include encoding categorical inputs, scaling numerical features, applying a consistent feature order, and generating failure probabilities using the trained ANN model.

Listing 6.1: Core inference pipeline logic from `inference.py`.

```
1 # 1) Validate and encode machine type using type_mapping
2 # 2) Scale numeric features using scaler.joblib
3 # 3) Reorder features using feature_order.json
4 # 4) Predict probability using failure_ann.keras
```

Chapter 7

Results, Evaluation, and Validation

7.1 Evaluation Metrics

To assess model performance comprehensively, several standard classification metrics were used:

- **Accuracy:** Overall correctness of predictions across both classes,
- **Precision:** Proportion of predicted failures that are actually failures,
- **Recall:** Sensitivity to actual failures—especially important in PdM, where missed detections can be costly,
- **F1-score:** Harmonic mean of precision and recall, providing a balanced view in imbalanced datasets,
- **ROC-AUC:** Threshold-independent metric that reflects the model’s ability to distinguish between failure and no-failure classes.

7.2 Baseline vs. ANN Comparison

The Random Forest model serves as a strong and interpretable baseline, offering reliable performance and feature-level insights. However, the ANN model demonstrates improved sensitivity to rare failure events. In particular, it achieves higher recall on the failure class—aligning with predictive maintenance goals where minimizing false negatives is critical.

7.2.1 Confusion Matrices

The confusion matrices below illustrate the distribution of prediction outcomes for both models. As expected, the ANN shows a better balance between correctly identified

failures and false positives, whereas the Random Forest tends to miss more failure cases.

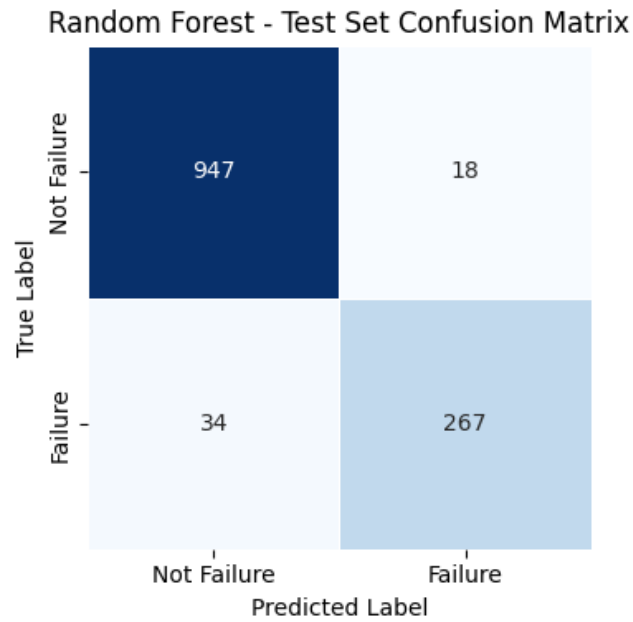


Figure 7.1: Confusion matrix for the Random Forest model on the test set. While it shows strong overall performance, several failure cases are misclassified as normal.

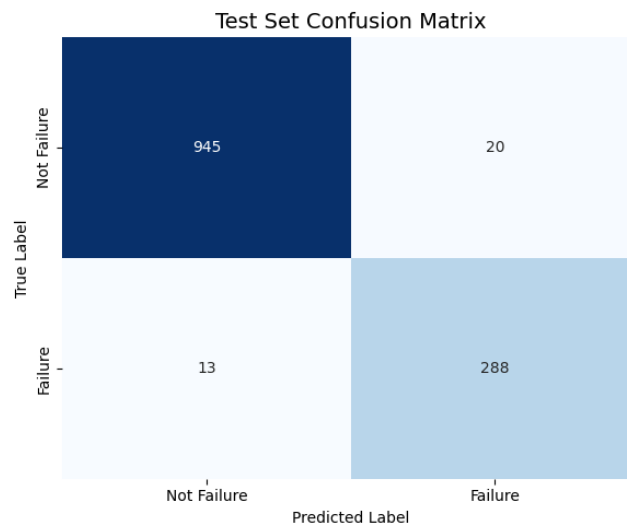


Figure 7.2: Confusion matrix for the ANN model on the test set. The model captures more true failures, with improved recall over the Random Forest baseline.

7.2.2 ROC Curves

ROC curves provide a threshold-independent view of classification performance. The figure below compares the ROC curves for both models, confirming that the ANN

achieves a higher AUC, reflecting better discrimination between failure and non-failure cases.

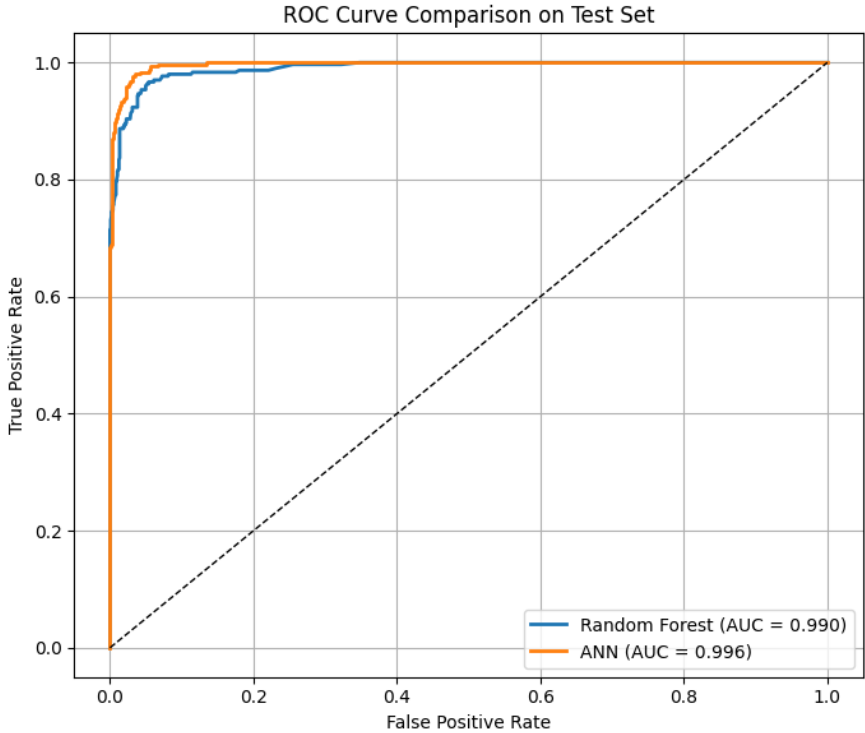


Figure 7.3: ROC curves for the ANN and Random Forest models on the test set. The ANN achieves a higher Area Under the Curve (AUC), indicating superior classification performance across different thresholds.

7.2.3 Performance Summary Table

Table 7.1: Performance Summary on Test Set

Model	Accuracy	Precision (Fail)	Recall (Fail)	F1 (Fail)	ROC-AUC
Random Forest	0.9589	0.9368	0.8870	0.9113	0.990
ANN (Deep)	0.9739	0.9351	0.9568	0.9458	0.996

7.3 Validation Strategy

To assess model generalization, a held-out test set was used during evaluation. All preprocessing steps—such as scaling, encoding, and feature ordering—were applied to the training data alone and then consistently reused on the test data to avoid data leakage.

Given the cost of undetected failures in industrial environments, special attention was placed on maximizing recall for the failure class. High recall ensures that the system detects as many true failures as possible, reducing the risk of unexpected breakdowns.

7.4 Benchmarking with Literature

While direct comparisons across studies can be challenging due to differences in datasets, features, and labeling strategies, it is still useful to situate this project’s results within the broader predictive maintenance research landscape.

Table 7.2 provides a high-level benchmarking summary, emphasizing the novelty of this work in delivering both strong modeling performance and a deployable interface. These comparisons should be interpreted cautiously, as many reviewed studies focus on isolated modeling results without addressing deployment or usability.

Table 7.2: Contextual benchmarking against selected predictive maintenance literature. Due to differences in datasets and experimental setups, comparisons are indicative only.

Study	Approach	Notes
Carvalho et al. [1]	Review	Summarizes machine learning methods used in PdM and reports typical performance ranges across industries.
Matzka [3]	XAI for PdM	Discusses the importance of model explainability and interpretable outputs in industrial deployments.
This project	RF + Deep ANN + Streamlit UI	Combines classical and deep learning models with a fully deployable inference interface, highlighting end-to-end feasibility.

Chapter 8

Deployment: Streamlit User Interface and Batch Inference

8.1 Deployment Goals

A key objective of this project is to demonstrate that predictive maintenance models can be integrated into user-friendly tools suitable for real-world environments. To that end, a Streamlit-based dashboard was developed, allowing users to interact with the model both in real time and through batch processing.

The interface supports:

- Real-time single-instance predictions based on manual input,
- Batch scoring through CSV file uploads,
- Adjustable decision thresholds, enabling the system to align with different risk tolerances and maintenance policies.

8.2 User Interface Overview

The dashboard is designed with usability in mind, making it accessible for both engineers and non-technical stakeholders. It offers:

- Dropdown selection for machine type (Low, Medium, High),
- Input fields for entering sensor values manually,
- Real-time failure probability output,
- A slider to adjust the decision threshold for flagging failure,
- File upload support for batch predictions, with downloadable results.

 Prediction Settings

Failure decision threshold 

0.50

Recommended thresholds

- 0.3 → Conservative (early warning)
- 0.5 → Balanced
- 0.7 → Strict (only critical failures)



Predictive Maintenance System

This tool predicts **potential machine failure** based on real-time sensor readings from a milling machine.

- Output: Failure Probability
- Model: ANN (binary classification)

 Machine Sensor Inputs

Machine Type

L 

Rotational Speed (rpm)

1500.00  

Air Temperature (K)

300.00  

Torque (Nm)

40.00  

Process Temperature (K)

310.00  

Tool Wear (min)

100.00  

 Predict Failure

 Batch Prediction via CSV Upload

Upload CSV file (exact 6-column schema required)

 Drag and drop file here
Limit 200MB per file • CSV

Browse files

Model trained on milling machine sensor data • Use as decision support, not as sole safety system.

Figure 8.1: Main screen of the Streamlit dashboard. Users can enter individual sensor values, adjust the decision threshold, or upload a CSV file for batch prediction. Results are presented interactively and can be downloaded for further analysis.

23

8.3 Single Prediction Workflow

When using the single prediction mode, the user provides input values and clicks the *Predict Failure* button. The application then:

1. Validates and formats the inputs,
2. Applies encoding and scaling using the saved training artifacts,
3. Passes the processed data to the ANN model for inference,
4. Compares the predicted failure probability to the selected threshold to generate a final decision.

Predictive Maintenance System

This tool predicts potential machine failure based on real-time sensor readings from a milling machine.

- Output: Failure Probability
- Model: ANN (binary classification)

Machine Sensor Inputs

Machine Type	Rotational Speed (rpm)
L	1500.00
Air Temperature (K)	Torque (Nm)
300.00	40.00
Process Temperature (K)	Tool Wear (min)
310.00	100.00
 Predict Failure	

Prediction Result

Failure Probability

0.14%

 No Failure Detected

Figure 8.2: Result screen for a single prediction. The dashboard displays the computed failure probability and a binary decision (Fail / No Fail) based on the user-defined threshold.

8.4 Batch Prediction via CSV Upload

Batch prediction supports screening many operating points at once. The application:

- Enforces a strict expected schema (exact six input columns),
- Validates and standardizes column names,
- Applies preprocessing and predicts probabilities for all rows,
- Outputs a downloadable CSV with predictions.

Batch Prediction via CSV Upload

Upload CSV file (exact 6-column schema required)

 Drag and drop file here
Limit 200MB per file • CSV

Browse files

 test_batch_input.csv 213.0B ×

 Valid CSV schema detected

	Type	Air temperature	Process temperature	Rotational speed	Torque	Tool wear
0	L	300	310	1500	40	100
1	M	298.5	308	1450	42	120
2	H	302	312	1600	38	80
3	L	299	309	1700	45	150
4	M	301.5	311	1400	35	60

Batch Prediction Results

	Type	Air temperature	Process temperature	Rotational speed	Torque	Tool wear	failure_probabilit
0	L	300	310	1500	40	100	0.001
1	M	298.5	308	1450	42	120	0.000
2	H	302	312	1600	38	80	0.008
3	L	299	309	1700	45	150	0.001
4	M	301.5	311	1400	35	60	0.00000

Predicted Failures

0

 Download Results CSV

Figure 8.3: Streamlit batch prediction interface showing CSV upload, automated pre-processing, tabular prediction results, and the option to download the annotated output file.

8.5 Operational Thresholds and Decision Support

The Streamlit dashboard includes a threshold slider that allows users to adjust how conservative or aggressive the failure predictions are. This flexibility helps align the predictive maintenance system with the specific operational risk preferences of different manufacturing environments.

- **Lower thresholds** trigger early warnings, maximizing sensitivity to potential failures but potentially increasing false alarms.
- **Medium thresholds** offer a balanced approach between sensitivity and precision.
- **Higher thresholds** make the system more selective, reducing false positives but risking missed failures.

By adjusting this threshold, operators can tailor the decision output to match the plant's tolerance for downtime, repair costs, and alert fatigue.

8.6 How to Run the Application (Reproducibility)

To reproduce the system and run the application locally, follow these steps:

1. Install the required Python packages using:

```
pip install -r requirements.txt
```

2. Launch the dashboard with the command:

```
streamlit run app.py
```

3. Ensure the `artifacts/` folder contains all the necessary serialized components:

- `failure_ann.keras` – trained ANN model,
- `scaler.joblib` – standardization model for numeric features,
- `feature_order.json` – consistent feature ordering,
- `type_mapping.json` – mapping for machine type encoding.

These steps ensure the deployed system behaves consistently with the training pipeline, supporting reliable and repeatable inference.

Chapter 9

Discussion, Limitations, Risks, and Ethics

9.1 Key Findings

This project demonstrates the feasibility and effectiveness of AI-driven predictive maintenance systems in manufacturing contexts. Key outcomes include:

- **Strong baseline performance:** Random Forest offered a robust and interpretable foundation, useful for initial deployment or benchmarking.
- **Improved recall through deep learning:** The ANN outperformed the baseline in detecting failure cases—an essential priority in PdM applications where false negatives are costly.
- **Deployment-readiness:** A streamlined Streamlit interface and reproducible pipeline confirmed that real-time or batch inference can be easily operationalized in an industrial setting.

9.2 Industrial Interpretation of Results

In predictive maintenance, the cost of an undetected failure typically outweighs that of a false alarm. Therefore, model selection and tuning should prioritize high recall, even at the expense of some precision. This project supports that principle by optimizing for failure sensitivity and ensuring the model generalizes well to unseen data.

9.3 Limitations

Despite its success, the project has several limitations:

- **Dataset scope:** The benchmark dataset, while useful for prototyping, may not capture the full variability of real-world factory conditions.
- **Sensor coverage:** The available features are limited compared to the richer sensor arrays found in modern industrial environments.
- **Generalization constraints:** Applying the model to different machine types or operating environments may require fine-tuning, retraining, or domain-specific calibration.

9.4 Risks and Mitigation

Several key risks were identified and addressed during development:

- **Overfitting:** Controlled through regularization techniques, dropout, early stopping, and validation monitoring.
- **Class imbalance:** Addressed using SMOTE and by emphasizing recall-focused metrics in evaluation.
- **Operational mismatches:** The threshold slider provides a practical control for aligning system alerts with on-site maintenance priorities.

9.5 Ethical Considerations

While predictive models can enhance maintenance planning, they are not intended to replace human expertise. Final decisions should always involve qualified engineers. To ensure responsible AI adoption, future deployments should incorporate explainability tools (e.g., SHAP) and maintain transparency in model behavior and limitations [3].

9.6 Future Work

Several directions could further enhance this system’s utility and robustness:

- Adding support for multi-class failure prediction within the user interface,
- Integrating explainability dashboards to improve trust and usability,
- Extending the system to real-time streaming environments (e.g., via MQTT or OPC UA),
- Exploring continual learning frameworks and drift detection for maintaining performance in live deployments.

Chapter 10

Conclusion

This project developed a complete AI-based predictive maintenance system tailored for milling machines. Both a deep Artificial Neural Network (ANN) and a Random Forest baseline were trained and evaluated using standard classification metrics, with particular focus on maximizing recall for the failure class—a critical factor in industrial applications.

To ensure deployment readiness, all preprocessing steps and model artifacts were saved, enabling a fully reproducible inference pipeline. A lightweight, interactive Streamlit dashboard was also implemented, supporting both real-time and batch predictions through an intuitive interface.

The results highlight that deep learning models offer improved sensitivity to rare failure events compared to classical methods. Combined with a user-friendly interface, the system demonstrates strong potential for practical integration into intelligent maintenance workflows in smart manufacturing environments.

Appendix A

Appendix A: Figures Checklist (Where to Insert Which Graph)

Chapter	Figures/Graphs to Insert
Ch. 4 Dataset	Feature histograms/boxplots; Machine Type distribution bar chart; Failure vs No-failure bar chart
Ch. 5 Preprocessing	Before vs After SMOTE class distribution; PCA PC1 vs PC2 scatter plot (colored by class/failure type)
Ch. 6 Modeling	System architecture diagram; ANN architecture diagram; ANN training curves (loss/accuracy)
Ch. 7 Results	Confusion matrix RF; Confusion matrix ANN; ROC curve comparison; (optional) feature importance plot for RF
Ch. 8 Deployment	Project folder structure screenshot; Streamlit UI screenshots (main, single prediction result, batch upload result)

Appendix B

Appendix B: Example CSV Schema for Batch Prediction

Include an example CSV header for batch inference:

```
1 Type,Air temperature [K],Process temperature [K],Rotational  
   speed [rpm],Torque [Nm],Tool wear [min]  
2 L,300.0,310.0,1500.0,40.0,100.0
```

Appendix C

Appendix C: Repository and Reproducibility Notes

C.1 GitHub Repository

All project code and resources are publicly available in a GitHub repository, ensuring full transparency and ease of replication. The repository includes data preprocessing scripts, model training notebooks, evaluation routines, deployment code, and saved artifacts.

- **GitHub Repository:**

<https://github.com/Soman10/Predictive-Maintenance-of-Machines---Case-Study-Intelligent-Systems-/tree/main>

C.2 Training and Experiment Reproducibility

Model training and evaluation were conducted on **Google Colab**, offering GPU support and a consistent development environment without requiring local setup.

- **Main Notebook:** `[Updated]_Predictive_Maintenance_CS_Project.ipynb`
- This notebook walks through data preprocessing, class balancing using SMOTE, model training (Random Forest and ANN), evaluation, and figure generation.
- Running the notebook cells sequentially will reproduce all results as shown in this report.

To ensure repeatability, random seeds are set where applicable.

C.3 Deployment Reproducibility

The deployment interface is built using **Streamlit**, allowing for real-time and batch predictions through a browser-based dashboard.

Key components include:

- `app.py`: Main Streamlit interface,
- `inference.py`: Handles preprocessing and model inference,
- `artifacts/`: Folder containing saved model files, encoders, and configuration objects,
- `requirements.txt`: Lists required Python packages.

To run the application locally:

```
pip install -r requirements.txt
streamlit run app.py
```

C.4 Software and Environment Details

The system was built and tested in the following software environment:

- **Programming Language:** Python 3.10+
- **Libraries Used:** TensorFlow (Keras), scikit-learn, imbalanced-learn, NumPy, pandas, matplotlib, seaborn
- **Deployment Framework:** Streamlit
- **Execution Platforms:** Google Colab (training) and local machine (deployment)
- **Operating System:** Cross-platform; tested on Windows and Linux

C.5 Reproducibility Statement

Every model, figure, and result presented in this report can be reproduced using the provided GitHub repository and training notebook. The codebase is structured modularly and extensively documented, ensuring transparency, traceability, and ease of future adaptation or extension.

Bibliography

- [1] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, and S. G. Alcalá, “A systematic literature review of machine learning methods applied to predictive maintenance,” *Computers & Industrial Engineering*, vol. 137, p. 106024, 2019.
- [2] G. Biau and E. Scornet, “A random forest guided tour,” *TEST*, vol. 25, no. 2, pp. 197–227, 2016.
- [3] S. Matzka, “Explainable artificial intelligence for predictive maintenance applications,” in *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*, pp. 69–74, IEEE, 2020.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [5] UCI Machine Learning Repository, “Ai4i 2020 predictive maintenance dataset (uci machine learning repository),” 2020. Dataset used for PdM modeling of milling machines.